



Figure 3: Deep learning with SystemML

A detailed reference to PyDML and DML is available in the official documentation at <https://apache.github.io/systemml/dml-language-reference.html>.

For the benefit of Python users, SystemML has several language-level APIs, which enable you to use it without needing to know DML or PyDML.

```
import systemml as sml
import numpy as np
m1 = sml.matrix(np.ones((3,3)) + 2)
m2 = sml.matrix(np.ones((3,3)) + 3)
m2 = m1 * (m2 + m1)
m4 = 1.0 - m2
m4.sum(axis=1).toNumPy()
```

Calling SystemML algorithms

SystemML has a sub-package called `mllearn`, which enables Python users to call SystemML algorithms. This is done with Scikit-learn or the `MLPipeline` API.

A sample code snippet for linear regression is shown below:

```
import numpy as np
from sklearn import datasets
from systemml.mllearn import LinearRegression

#1 Load the diabetes dataset
diabetes = datasets.load_diabetes()

# 2 Use only one feature
diabetes_X = diabetes.data[:, np.newaxis, 2]

#3 Split the data into training/testing sets
X_train = diabetes_X[:-20]
X_test = diabetes_X[-20:]

#4 Split the targets into training/testing sets
y_train = diabetes.target[:-20]
y_test = diabetes.target[-20:]

#5 Create linear regression object
regr = LinearRegression(spark, fit_intercept=True,
C=float("inf"), solver='direct-solve')

#6 Train the model using the training sets
regr.fit(X_train, y_train)
```

```
y_predicted = regr.predict(X_test)
print('Residual sum of squares: %.2f' % np.mean((y_
predicted - y_test) ** 2))
```

The output of the above code is shown below:

```
Residual sum of squares: 6991.17
```

A sample code snippet with the `MLPipeline` interface and the logistic regression is shown below:

```
# MLPipeline way
from pyspark.ml import Pipeline
from systemml.mllearn import LogisticRegression
from pyspark.ml.feature import HashingTF, Tokenizer

training = spark.createDataFrame([
    (0, "a b c d e spark", 1.0),
    (1, "b d", 2.0),
    (2, "spark f g h", 1.0),
    (3, "hadoop mapreduce", 2.0),
    (4, "b spark who", 1.0),
    (5, "g d a y", 2.0),
    (6, "spark fly", 1.0),
    (7, "was mapreduce", 2.0),
    (8, "e spark program", 1.0),
    (9, "a e c l", 2.0),
    (10, "spark compile", 1.0),
    (11, "hadoop software", 2.0)
], ["id", "text", "label"])

tokenizer = Tokenizer(inputCol="text", outputCol="words")
hashingTF = HashingTF(inputCol="words",
outputCol="features", numFeatures=20)
lr = LogisticRegression(sqlCtx)
pipeline = Pipeline(stages=[tokenizer, hashingTF, lr])
model = pipeline.fit(training)

test = spark.createDataFrame([
    (12, "spark i j k"),
    (13, "l m n"),
    (14, "mapreduce spark"),
    (15, "apache hadoop")], ["id", "text"])

prediction = model.transform(test)
prediction.show()
```

Deep learning with SystemML

Deep learning has evolved into a specialised class of machine learning algorithms, which makes handling of features simple and efficient. SystemML also has support for deep learning. There are three methods with which deep learning can be carried out in SystemML (Figure 3):

- With the help of the DML-bodied NN library. This enables the utilisation of DML to implement neural networks.